

## **Manual Tecnico**

Nicolas Mora, Yessica Velásquez

Universidad de Cundinamarca

Ingeniería de software II

German Ortiz Diaz

16 de septiembre de 2026

# UCampus: Manual técnico

*Plataforma de difusión universitaria · Universidad de Cundinamarca*

Autores: Nicolas Mora, Yessica Velasquez · Ingeniería de Software II · Docente:

German Ortiz Diaz · Versión 0.2 · Octubre de 2026

## 1. Introducción

Este manual describe la arquitectura, el modelo de datos, la API y el procedimiento de instalación y operación de UCampus, una plataforma web que centraliza eventos, convocatorias y noticias de la comunidad universitaria. Está dirigido a quienes instalen, mantengan o amplíen el sistema.

La versión 0.2 incluye: inicio de sesión por rol, registro abierto de estudiantes, publicaciones (eventos, convocatorias y noticias), interacción (me gusta, guardar, comentar, inscribirse) y administración de usuarios y roles.

Este manual sigue la planeación Scrum del proyecto (5 épicas y 5 sprints) e indica, historia por historia, qué criterios de aceptación cumple la versión 0.2 y cuáles siguen pendientes.

## 2. Contexto del proyecto y actores

UCampus nace porque los estudiantes se enteran de eventos, convocatorias y noticias por redes sociales, correos y grupos digitales, y la información queda dispersa. El informe del proyecto cita que 9 de cada 10 estudiantes han experimentado algún nivel de infoxicación digital (Rojas Méndez y Rodríguez Ayala, 2024).

## 2.1 Objetivos

**Objetivo general:** desarrollar una plataforma web universitaria que centralice y facilite la consulta de información sobre eventos, actividades extracurriculares, convocatorias y novedades de interés para los estudiantes.

1. Identificar las necesidades del plantel educativo sobre difusión y consulta de información mediante entrevistas.
2. Diseñar una interfaz web sencilla e intuitiva para consultar y publicar información.
3. Desarrollar las funcionalidades principales de UCampus de manera incremental.

## 2.2 Actores y roles del sistema

Los requerimientos se levantaron con entrevistas a estudiantes, bienestar educativo y administradores. La tabla muestra qué rol del sistema usa cada stakeholder del informe.

| Stakeholder                    | Tipo          | Rol en UCampus | Qué hace en la plataforma                                               |
|--------------------------------|---------------|----------------|-------------------------------------------------------------------------|
| Estudiantes                    | Usuario final | ESTUDIANTE     | Consulta, busca, comenta, guarda e inscribe                             |
| Bienestar Universitario        | Interno       | BIENESTAR      | Publica y gestiona sus publicaciones                                    |
| Oficina de Comunicaciones      | Interno       | BIENESTAR      | Sin rol propio en la versión 0.2; publica con los permisos de Bienestar |
| Administrador de la plataforma | Interno       | ADMIN          | Administra usuarios, roles y todas las                                  |

| Stakeholder | Tipo               | Rol en UCampus | Qué hace en la plataforma |
|-------------|--------------------|----------------|---------------------------|
|             |                    |                | publicaciones             |
| Docentes    | Usuario secundario | DOCENTE        | Consulta, busca y comenta |

### 3. Gestión del proyecto con Scrum

El desarrollo se organizó con Scrum en Jira: 5 épicas, una por sprint, con 54 actividades entre historias y tareas. Los sprints van del 2 de septiembre al 3 de noviembre de 2026.

#### 3.1 Épicas

| Épica                                     | Clave Jira | Historias de usuario | Sprint |
|-------------------------------------------|------------|----------------------|--------|
| E1 Gestión de usuarios y acceso           | SCRUM-6    | HU01, HU02           | 1      |
| E2 Gestión de publicaciones               | SCRUM-7    | HU03, HU04, HU05     | 2      |
| E3 Consulta y búsqueda de información     | SCRUM-8    | HU06, HU07, HU08     | 3      |
| E4 Información de eventos y convocatorias | SCRUM-9    | HU09, HU10           | 4      |
| E5 Interacción de los estudiantes         | SCRUM-10   | HU11                 | 5      |

### 3.2 Sprints

El estado corresponde al tablero de Jira en la captura incluida en el informe.

| Sprint | Fechas             | Épica | Historias<br>(clave Jira)                                                            | Actividades | Estado en<br>Jira               |
|--------|--------------------|-------|--------------------------------------------------------------------------------------|-------------|---------------------------------|
| 1      | 2 sep – 16<br>sep  | E1    | HU01<br><br>(SCRUM-12),<br><br>HU02<br><br>(SCRUM-13)                                | 10          | Todas<br><br>finalizadas        |
| 2      | 17 sep – 8<br>oct  | E2    | HU03<br><br>(SCRUM-14),<br><br>HU04<br><br>(SCRUM-15),<br><br>tareas de<br><br>HU05  | 13          | Activo, tareas<br><br>por hacer |
| 3      | 9 oct – 16 oct     | E3    | HU06<br><br>(SCRUM-22),<br><br>HU07<br><br>(SCRUM-20),<br><br>HU08<br><br>(SCRUM-21) | 15          | Por iniciar                     |
| 4      | 19 oct – 26<br>oct | E4    | HU09<br><br>(SCRUM-23),<br><br>HU10<br><br>(SCRUM-24)                                | 10          | Por iniciar                     |
| 5      | 27 oct – 3         | E5    | HU11                                                                                 | 6           | Por iniciar                     |

| Sprint | Fechas | Épica | Historias<br>(clave Jira) | Actividades | Estado en<br>Jira |
|--------|--------|-------|---------------------------|-------------|-------------------|
|        | nov    |       | (SCRUM-25)                |             |                   |

### 3.3 Tareas por historia de usuario

| Historia               | Tareas (clave Jira)                                                                                                                                                                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HU01 Iniciar sesión    | Crear la tabla de usuarios en la base de datos (SCRUM-28) · Diseñar la interfaz de inicio de sesión (SCRUM-29) · Conectar la interfaz con la base de datos (SCRUM-30) · Validar usuario y contraseña (SCRUM-31) · Mostrar el acceso según el rol del usuario (SCRUM-32) |
| HU02 Gestionar roles   | Agregar el campo de rol a los usuarios (SCRUM-33) · Crear los roles estudiante y administrador (SCRUM-34) · Mostrar las opciones correspondientes a cada rol (SCRUM-35)                                                                                                 |
| HU03 Crear publicación | Diseñar la interfaz para crear publicaciones (SCRUM-36) · Agregar los campos necesarios (SCRUM-37) · Agregar las categorías: eventos, actividades, convocatorias, noticias (SCRUM-38) · Guardar la publicación en la base de datos (SCRUM-39) · Mostrar la publicación  |

| Historia                     | Tareas (clave Jira)                                                                                                                                                                                       |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              | después de guardarla (SCRUM-40)                                                                                                                                                                           |
| HU04 Editar publicación      | Mostrar las publicaciones que pueden ser editadas (SCRUM-41) · Crear el botón Editar (SCRUM-42) · Permitir modificar la información (SCRUM-43) · Guardar y mostrar los cambios realizados (SCRUM-44)      |
| HU05 Eliminar publicación    | Mostrar un mensaje de confirmación (SCRUM-47) · Actualizar la lista de publicaciones (SCRUM-49). Las demás tareas de HU05 no aparecen en la captura                                                       |
| HU06 Consultar publicaciones | Crear la página de publicaciones (SCRUM-50) · Mostrar las publicaciones realizadas (SCRUM-51) · Crear la opción Ver detalles (SCRUM-52) · Organizar la información de forma clara (SCRUM-53)              |
| HU07 Buscar información      | Crear la barra de búsqueda (SCRUM-54) · Buscar el término en las publicaciones (SCRUM-55) · Mostrar las publicaciones encontradas (SCRUM-56) · Mostrar un mensaje cuando no existan resultados (SCRUM-57) |
| HU08 Filtrar publicaciones   | Crear el selector de categorías (SCRUM-58) · Agregar las categorías disponibles (SCRUM-59) · Mostrar las publicaciones                                                                                    |

| Historia                             | Tareas (clave Jira)                                                                                                                                                                                                                                                                 |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                      | según la categoría seleccionada (SCRUM-60) · Probar el funcionamiento de los filtros (SCRUM-61)                                                                                                                                                                                     |
| HU09 Consultar detalles de un evento | Crear la vista de detalles del evento (SCRUM-62) · Mostrar fecha y hora (SCRUM-63) · Mostrar el lugar (SCRUM-64) · Organizar la información para facilitar su consulta (SCRUM-65)                                                                                                   |
| HU10 Consultar convocatorias         | Crear la vista de detalles de la convocatoria (SCRUM-66) · Mostrar la descripción y la fecha límite (SCRUM-67) · Mostrar los requisitos (SCRUM-68) · Probar la consulta de convocatorias (SCRUM-69)                                                                                 |
| HU11 Comentar publicaciones          | Crear la tabla de comentarios (SCRUM-70) · Crear el espacio para escribir un comentario (SCRUM-71) · Validar que el comentario no esté vacío (SCRUM-72) · Guardar y asociar el comentario a la publicación (SCRUM-73) · Mostrar los comentarios debajo de la publicación (SCRUM-74) |

### 3.4 Planeación inicial (diagrama de Gantt)

El Gantt del informe es la planeación de alto nivel por fases; el detalle del desarrollo se lleva en Jira por sprints.

| Fase          | Inicio | Fin    | Duración | Contenido y avance en la captura                                                                                   |
|---------------|--------|--------|----------|--------------------------------------------------------------------------------------------------------------------|
| 1. Análisis   | 11 ago | 14 ago | 4 días   | Encuesta, problema, objetivos y requisitos al 100 %; stakeholders al 60 %; clasificación de requerimientos al 70 % |
| 2. Diseño     | 17 ago | 24 ago | 6 días   | Arquitectura (5 días), base de datos con MER, diseño UML, casos de uso, diagrama de clases e interfaz; 0 %         |
| 3. Desarrollo | 25 ago | 7 sep  | 10 días  | Integración de módulos (7 días) y panel administrativo (3 días); 0 %                                               |

| Fase          | Inicio    | Fin       | Duración | Contenido y avance en la captura                                      |
|---------------|-----------|-----------|----------|-----------------------------------------------------------------------|
| 4. Pruebas    | Sin fecha | Sin fecha | Sin dato | Pruebas funcionales, pruebas de interfaz y corrección de errores; 0 % |
| 5. Despliegue | Sin fecha | Sin fecha | Sin dato | 0 %                                                                   |

## 4. Arquitectura general

UCampus sigue una arquitectura cliente-servidor de tres capas:

- **Presentación:** una sola página (`public/index.html`) en HTML, CSS y JavaScript sin frameworks. La sirve el mismo servidor.
- **Lógica:** servidor Node.js con Express que expone una API REST en JSON bajo la ruta `/api`. Valida autenticación, permisos por rol y datos de entrada.
- **Datos:** base de datos MySQL 8 con llaves foráneas.

**Flujo de una petición:** el navegador envía la petición con el encabezado

`Authorization: Bearer <token>`. El middleware `auth` verifica el token JWT y el rol. La ruta ejecuta consultas parametrizadas contra MySQL y responde en JSON.

| Componente    | Tecnología              | Uso                           |
|---------------|-------------------------|-------------------------------|
| Servidor      | Node.js 18+ y Express 4 | API REST y archivos estáticos |
| Base de datos | MySQL 8 con mysql2 3    | Pool de 10 conexiones         |

| Componente    | Tecnología     | Uso                          |
|---------------|----------------|------------------------------|
| Contraseñas   | bcryptjs 2     | Hash con costo 10            |
| Sesiones      | jsonwebtoken 9 | Token JWT firmado, 8 horas   |
| Configuración | dotenv 16      | Variables de entorno en .env |

## 5. Requisitos e instalación

**Requisitos:** Node.js 18 o superior, MySQL 8 y un navegador moderno.

```
mysql -u root -p < schema.sql
cp .env.example .env      (editar con la clave de MySQL y un JWT_SECRET largo)
npm install
npm run seed              (opcional: usuarios y publicaciones de ejemplo)
npm start
```

Luego se entra a `http://localhost:3000`. Al arrancar, el servidor conecta con la base y asegura los datos base (sección 11, regla 7). Si no puede conectarse, imprime el motivo y se detiene.

## 6. Estructura del proyecto

```
ucampus-app/
├── server.js      API, autenticación, permisos y reglas de negocio
├── seed.js        datos de demostración (usuarios, publicaciones)
├── schema.sql     esquema de la base de datos
├── package.json   dependencias y scripts (start, seed)
├── .env.example   plantilla de configuración
├── README.md      guía rápida
├── public/
│   └── index.html interfaz completa (login, feed, administración)
```

## 7. Configuración

Las variables se definen en el archivo `.env` (nunca debe subirse al repositorio; ya está en `.gitignore`).

| Variable | Valor por defecto | Descripción      |
|----------|-------------------|------------------|
| DB_HOST  | localhost         | Servidor MySQL   |
| DB_USER  | root              | Usuario de MySQL |

| Variable    | Valor por defecto | Descripción                                                                     |
|-------------|-------------------|---------------------------------------------------------------------------------|
| DB_PASSWORD | (vacío)           | Clave de MySQL                                                                  |
| DB_NAME     | ucampus           | Nombre de la base de datos                                                      |
| JWT_SECRET  | (obligatoria)     | Clave de firma de tokens, mínimo 16 caracteres. Sin ella el servidor no arranca |
| PORT        | 3000              | Puerto HTTP                                                                     |

## 8. Modelo de datos

### 8.1 Tablas en uso

| Tabla                | Propósito                                                                | Relaciones principales           |
|----------------------|--------------------------------------------------------------------------|----------------------------------|
| roles                | Catálogo de roles                                                        | 1 rol → N usuarios               |
| usuarios             | Cuentas, credenciales y datos personales                                 | Pertenece a un rol               |
| programas_academicos | Programas de pregrado; la facultad se guarda en <code>descripcion</code> | 1 programa → N estudiantes       |
| estudiantes          | Datos académicos de usuarios con rol estudiante                          | 1 a 1 con usuarios               |
| categorias_evento    | Categorías de publicaciones                                              | 1 categoría → N publicaciones    |
| publicaciones        | Unidad del feed: título, contenido, tipo, categoría y autor              | Opcionalmente enlaza a un evento |

| Tabla                   | Propósito                                              | Relaciones principales                |
|-------------------------|--------------------------------------------------------|---------------------------------------|
| eventos                 | Detalle de eventos y convocatorias: fecha, lugar, cupo | 0 o 1 por publicación                 |
| comentarios             | Comentarios en publicaciones                           | Publicación y usuario                 |
| reacciones              | Me gusta                                               | Llave compuesta usuario + publicación |
| publicaciones_guardadas | Publicaciones guardadas por el usuario                 | Llave compuesta usuario + publicación |
| inscripciones_evento    | Inscripciones y postulaciones                          | Evento y estudiante, única por pareja |

## 8.2 Ajustes respecto al diagrama ER original

- `publicaciones` incorpora `tipo` (evento, convocatoria o noticia), `titulo`, `id_categoria` e `id_evento` (opcional). Así `eventos` guarda solo lo propio del evento y no se duplican título, descripción ni categoría.
- `eventos_guardados` pasó a `publicaciones_guardadas`, para poder guardar también noticias.
- Se creó la tabla `reacciones` para los me gusta.
- `eventos` agrega el campo `lugar` (texto libre).

## 8.3 Tablas reservadas

`edificios`, `pisos` y `espacios` existen en el esquema, pero la interfaz aún no las usa (el lugar de un evento es texto libre). Las tablas `materias`, `horarios` y `estudiante_materia` del diagrama original no están creadas en esta versión.

## 8.4 Estados

- `publicaciones.estado`: activa o eliminada. La eliminación es lógica: el registro se conserva.
- `comentarios.estado`: visible u oculto (aún sin pantalla de moderación).
- `inscripciones_evento.estado`: confirmada o cancelada.
- `eventos.estado`: programado, cancelado o finalizado (aún sin uso en la interfaz).

## 9. Autenticación y autorización

### 9.1 Inicio de sesión

`POST /api/auth/login` recibe `usuario` (correo o nombre de usuario) y `password`. Si son correctos, responde con un token JWT válido por 8 horas y los datos básicos del usuario (id, nombre y rol). Con credenciales incorrectas responde 401 con un mensaje genérico, sin indicar cuál dato falló.

El frontend guarda el token en `sessionStorage`, de modo que la sesión termina al cerrar la pestaña. Si el servidor responde 401 en cualquier petición, el frontend cierra la sesión y vuelve al login.

### 9.2 Registro abierto

Como no se dispone de cuentas institucionales, `POST /api/auth/registro` permite crear una cuenta con cualquier correo. Reglas:

- Se requieren nombres, apellidos, correo válido, programa académico existente y contraseña de 8 a 72 caracteres.
- El correo no puede estar repetido (409 si ya existe).
- El rol siempre es ESTUDIANTE y lo fija el servidor; el cliente no puede elegirlo.

- El nombre de usuario se deriva del correo y se hace único agregando dígitos si es necesario.
- Se crean en una misma transacción la fila de `usuarios` y la de `estudiantes`.
- Al terminar, la respuesta incluye un token, por lo que la persona entra directamente.

### 9.3 Roles y permisos

| Acción                                                             | Estudiante | Docente | Bienestar      | Admin |
|--------------------------------------------------------------------|------------|---------|----------------|-------|
| Consultar,<br>buscar, filtrar,<br>guardar, dar me gusta y comentar | Sí         | Sí      | Sí             | Sí    |
| Inscribirse o postularse                                           | Sí         | No      | No             | No    |
| Crear publicaciones                                                | No         | No      | Sí             | Sí    |
| Editar o eliminar                                                  | No         | No      | Solo las tuyas | Todas |
| Gestionar usuarios y roles                                         | No         | No      | No             | Sí    |

El rol de una cuenta nueva solo cambia cuando un administrador lo modifica desde la pestaña Usuarios. Un administrador no puede cambiar su propio rol. Los permisos se validan siempre en el servidor; ocultar botones en la interfaz es solo comodidad.

## 10. API REST

Todas las rutas devuelven JSON. Los errores tienen la forma `{ error: mensaje }`. La columna Acceso indica el rol requerido; Autenticado significa cualquier rol con sesión iniciada.

| Método | Ruta                               | Acceso           | Descripción                                                |
|--------|------------------------------------|------------------|------------------------------------------------------------|
| POST   | /api/auth/login                    | Público          | Inicia sesión                                              |
| POST   | /api/auth/registro                 | Público          | Crea cuenta de estudiante                                  |
| GET    | /api/programas                     | Público          | Lista programas activos con su facultad                    |
| GET    | /api/auth/me                       | Autenticado      | Datos de la sesión actual                                  |
| GET    | /api/categorias                    | Autenticado      | Lista categorías activas                                   |
| GET    | /api/publicaciones                 | Autenticado      | Lista publicaciones (hasta 100, las más recientes primero) |
| POST   | /api/publicaciones                 | Bienestar, Admin | Crea una publicación                                       |
| PUT    | /api/publicaciones/:id             | Bienestar, Admin | Edita una publicación                                      |
| DELETE | /api/publicaciones/:id             | Bienestar, Admin | Elimina (lógicamente) una publicación                      |
| POST   | /api/publicaciones/:id/like        | Autenticado      | Alterna el me gusta                                        |
| POST   | /api/publicaciones/:id/guardar     | Autenticado      | Alterna guardar                                            |
| POST   | /api/publicaciones/:id/inscripcion | Estudiante       | Alterna la inscripción                                     |
| GET    | /api/publicaciones/:id/            | Autenticado      | Lista comentarios                                          |

| Método | Ruta                                   | Acceso      | Descripción                 |
|--------|----------------------------------------|-------------|-----------------------------|
|        | comentarios                            |             | visibles                    |
| POST   | /api/publicaciones/:id/<br>comentarios | Autenticado | Agrega un comentario        |
| GET    | /api/usuarios                          | Admin       | Lista usuarios activos      |
| GET    | /api/roles                             | Admin       | Lista nombres de roles      |
| PATCH  | /api/usuarios/:id/rol                  | Admin       | Cambia el rol de un usuario |

**Filtros de GET /api/publicaciones:** tipo (evento, convocatoria o noticia), categoria (nombre exacto) y q (busca en título y contenido). Cada publicación devuelve además el conteo de me gusta, comentarios e inscritos, y tres indicadores del usuario actual: liked, guardado e inscrito.

**Campos para crear o editar:** tipo, categoria, titulo (3 a 150 caracteres), contenido, fecha (obligatoria si no es noticia), lugar y cupo (opcional; 0 o vacío significa sin cupo).

**Códigos de respuesta:** 200 y 201 éxito; 400 datos inválidos; 401 sesión ausente o vencida; 403 rol sin permiso; 404 recurso inexistente; 409 conflicto (correo repetido o cupo lleno); 429 demasiados intentos; 500 error del servidor.

## 11. Reglas de negocio

1. **Publicar.** Un evento o convocatoria crea una fila en eventos y otra en publicaciones, dentro de una transacción. Una noticia crea solo la publicación.
2. **Editar.** No permite cambiar el tipo. Bienestar solo modifica sus propias publicaciones; el administrador, todas.

3. **Eliminar.** Cambia el estado a eliminada. La publicación deja de mostrarse, pero sus datos se conservan.
4. **Inscripción.** Solo para estudiantes y sobre publicaciones con evento asociado. Alterna entre inscrito y cancelado. Se ejecuta en una transacción con bloqueo de la fila del evento (`FOR UPDATE`), de modo que dos inscripciones simultáneas no superen el cupo. Si el cupo está lleno responde 409. Reinscribirse reactiva la fila anterior.
5. **Me gusta y guardar.** Cada llamada alterna el estado; la llave compuesta evita duplicados.
6. **Comentarios.** No se aceptan vacíos ni de más de 1000 caracteres.
7. **Datos base.** En cada arranque el servidor asegura los cuatro roles (ESTUDIANTE, DOCENTE, BIENESTAR y ADMIN) y sincroniza el catálogo de 17 programas de pregrado por su código, con la facultad en `descripcion`. Las categorías se crean solo si la tabla está vacía. El seed de demostración se ejecuta únicamente si no existen usuarios.

## 12. Frontend

`public/index.html` contiene toda la interfaz. Puntos clave:

- **Estado global:** el objeto `S` guarda sesión, pestaña activa, filtros, publicaciones y comentarios cargados.
- **Ciente de API:** la función `api()` agrega el token, interpreta errores y cierra la sesión ante un 401. Si la página se abre fuera de Express (por ejemplo, con Live Server), apunta a `http://localhost:3000`.
- **Renderizado:** las funciones `nav`, `stories`, `feed`, `side` y `post` generan el HTML. Todo texto proveniente de la base se pasa por `esc()` para evitar inyección de HTML.
- **Eventos:** un único manejador de clics delega las acciones según atributos `data-*`.
- **Búsqueda:** se consulta al servidor con un retraso de 300 ms tras dejar de escribir.

- **Pestañas:** Inicio, Eventos, Convocatorias, Noticias, Guardados (filtrada en el cliente) y Usuarios (solo administrador).
- **Presentación:** diseño responsivo (barra lateral en escritorio, barra inferior en celular), modo claro y oscuro, y paleta verde y dorado institucional.

### 13. Seguridad

| Riesgo                  | Medida aplicada                                                                          |
|-------------------------|------------------------------------------------------------------------------------------|
| Inyección SQL           | Todas las consultas usan parámetros (?)                                                  |
| Robo de contraseñas     | Almacenadas con bcrypt; nunca se devuelven al cliente                                    |
| Acceso no autorizado    | JWT verificado en el servidor y validación de rol por ruta                               |
| Escalada de privilegios | El rol del registro lo fija el servidor; el cambio de rol es exclusivo del administrador |
| Fuerza bruta            | Límite de 8 intentos por IP cada 10 minutos en login y registro                          |
| XSS                     | Escape de HTML en toda salida dinámica                                                   |
| Acceso entre orígenes   | CORS permitido solo para localhost en desarrollo                                         |

**Limitaciones conocidas:** no hay HTTPS (debe configurarse al desplegar); el límite de intentos vive en memoria y se reinicia con el servidor; no hay verificación del correo ni recuperación de contraseña; el token no puede revocarse antes de que venza.

### 14. Datos de prueba

La clave de todos los usuarios sembrados es `Ucampus2026*`. Debe cambiarse o eliminarse antes de cualquier despliegue real.

| Rol           | Correo                                                                      |
|---------------|-----------------------------------------------------------------------------|
| Administrador | admin@ucundinamarca.edu.co                                                  |
| Bienestar     | bienestar@ucundinamarca.edu.co y<br>deportes@ucundinamarca.edu.co           |
| Docente       | camilo.pena@ucundinamarca.edu.co                                            |
| Estudiante    | valentina.rojas@ucundinamarca.edu.co y<br>laura.mendez@ucundinamarca.edu.co |

## 15. Solución de problemas

| Síntoma                                  | Causa probable                                    | Solución                                                                  |
|------------------------------------------|---------------------------------------------------|---------------------------------------------------------------------------|
| El servidor no inicia y pide JWT_SECRET  | Falta la variable en .env                         | Definirla con al menos 16 caracteres                                      |
| Dice que no se pudo conectar con la base | Credenciales erróneas o esquema no ejecutado      | Revisar .env y ejecutar schema.sql                                        |
| La lista de programas no carga           | Servidor detenido o versión anterior en ejecución | Abrir /api/programas;<br>reiniciar con npm start;<br>recargar con Ctrl+F5 |
| Error 401 en todo                        | Token vencido o JWT_SECRET cambiado               | Cerrar sesión e iniciar de nuevo                                          |
| Error 429 al iniciar sesión              | Más de 8 intentos en 10 minutos                   | Esperar o reiniciar el servidor en desarrollo                             |
| El seed no hace nada                     | Ya existen usuarios                               | Es el comportamiento esperado; recrear la base para volver a sembrar      |

## 16. Trazabilidad con los requerimientos

### 16.1 Historias de usuario

El estado en Jira es el de la captura del informe. El cumplimiento es el de la implementación v0.2, revisada contra los criterios de aceptación de cada historia.

| Historia               | Épica · Sprint | Estado en Jira | Implementación técnica                                                                                                                                | Cumplimiento v0.2 |
|------------------------|----------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| HU01 Iniciar sesión    | E1 · S1        | Finalizada     | POST<br><code>/api/auth/login</code><br>, pantalla de login y rol dentro del token                                                                    | Cumplido          |
| HU02 Gestionar roles   | E1 · S1        | Finalizada     | Pestaña Usuarios, GET<br><code>/api/usuarios</code> y PATCH<br><code>/api/usuarios/:id/rol</code> ; roles ESTUDIANTE y ADMIN, más DOCENTE y BIENESTAR | Cumplido          |
| HU03 Crear publicación | E2 · S2        | Por hacer      | POST<br><code>/api/publicaciones</code> , formulario con validación                                                                                   | Cumplido          |

| Historia                     | Épica · Sprint | Estado en Jira     | Implementación técnica                                                              | Cumplimiento v0.2                                             |
|------------------------------|----------------|--------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------|
|                              |                |                    | de campos obligatorios y categorías                                                 |                                                               |
| HU04 Editar publicación      | E2 · S2        | Por hacer          | PUT<br>/api/publicaciones/:id; el botón Editar solo aparece a quien puede modificar | Cumplido                                                      |
| HU05 Eliminar publicación    | E2 · S2        | Por hacer          | DELETE<br>/api/publicaciones/:id con confirmación; la lista se actualiza            | Cumplido                                                      |
| HU06 Consultar publicaciones | E3 · S3        | Sprint por iniciar | GET<br>/api/publicaciones; la tarjeta del feed muestra el detalle completo          | Cumplido; no hay pantalla aparte para Ver detalles (SCRUM-52) |
| HU07 Buscar información      | E3 · S3        | Sprint por iniciar | Parámetro q y mensaje cuando no hay resultados                                      | Cumplido                                                      |

| Historia                             | Épica · Sprint | Estado en Jira     | Implementación técnica                                                                         | Cumplimiento v0.2                                     |
|--------------------------------------|----------------|--------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| HU08 Filtrar publicaciones           | E3 · S3        | Sprint por iniciar | Parámetro categoria; tocar de nuevo la categoría quita el filtro                               | Cumplido                                              |
| HU09 Consultar detalles de un evento | E4 · S4        | Sprint por iniciar | Título, descripción, fecha, lugar y cupo en la tarjeta                                         | Parcial: no se captura ni se muestra la hora          |
| HU10 Consultar convocatorias         | E4 · S4        | Sprint por iniciar | Pestaña Convocatorias; descripción, fecha límite y postulación con cupo                        | Parcial: no se capturan ni se muestran los requisitos |
| HU11 Comentar publicaciones          | E5 · S5        | Sprint por iniciar | POST /api/publicaciones/:id/comentarios; exige sesión, rechaza vacíos y asocia el comentario a | Cumplido                                              |

|          |                |                |                           |                      |
|----------|----------------|----------------|---------------------------|----------------------|
| Historia | Épica · Sprint | Estado en Jira | Implementación<br>técnica | Cumplimiento<br>v0.2 |
|          |                |                | la publicación            |                      |

## 16.2 Requerimientos no funcionales

| Requerimiento                                           | Cumplimiento                       | Evidencia                                                                                                  |
|---------------------------------------------------------|------------------------------------|------------------------------------------------------------------------------------------------------------|
| RNF1 Interfaz sencilla e intuitiva                      | Cumplido, sin pruebas con usuarios | Diseño responsivo tipo red social, búsqueda y filtros por categoría; no se aplicaron pruebas de usabilidad |
| RNF2 Solo usuarios autorizados crean, editan o eliminan | Cumplido                           | Validación de rol en el servidor (403) y de autoría para Bienestar                                         |
| RNF3 Tiempo de respuesta adecuado                       | No medido                          | Consultas parametrizadas con índice por estado y fecha y resultados limitados a 100; sin pruebas de carga  |
| RNF4 Modificar y actualizar sin afectar el sistema      | Parcial                            | Capas separadas y llaves foráneas; sin pruebas automáticas ni migraciones de esquema                       |

## 17. Brechas, trabajo futuro y despliegue

### 17.1 Brechas frente a los criterios de aceptación

La versión 0.2 cumple 9 de las 11 historias por completo y 2 de forma parcial. Estas son las diferencias con el plan:

- **Hora del evento (HU09, SCRUM-63):** el formulario solo recoge la fecha y el sistema guarda las 12:00 por defecto. Falta un campo de hora y mostrarla en la tarjeta.
- **Requisitos de las convocatorias (HU10, SCRUM-68):** la columna `requisitos` existe en `eventos`, pero el formulario y la API aún no la usan.
- **Ver detalles (HU06, SCRUM-52):** el detalle completo se ve en la tarjeta del feed; no hay una página de detalle por publicación.
- **Actividades (HU03, SCRUM-38):** no tienen un tipo propio; se publican como evento y se distinguen por categoría.
- **Información desactualizada:** las publicaciones se pueden editar o eliminar, pero los eventos con fecha pasada no se ocultan solos.
- **Panel administrativo:** el Gantt lo prevé como módulo; en la versión 0.2 lo cubre la pestaña Usuarios, que lista usuarios y cambia roles, sin crear ni desactivar cuentas.

### 17.2 Trabajo futuro

**Pendiente:** verificación de correo y recuperación de contraseña, subida de imágenes, notificaciones sobre actividades de interés, uso de los espacios del campus en los eventos, materias y horarios, moderación de comentarios y pruebas automáticas.

### 17.3 Despliegue en producción

**Recomendaciones para producción:** servir detrás de un proxy inverso con HTTPS (por ejemplo Nginx), ejecutar el proceso con un gestor como pm2, usar un `JWT_SECRET` largo y

aleatorio, crear un usuario de MySQL con privilegios limitados a la base `ucampus`, programar copias de seguridad de la base y reemplazar el límite de intentos en memoria por uno compartido si se usan varias instancias.

## 18. Referencias y repositorio

- Repositorio del proyecto: <https://github.com/natt134/Ucampus.git>
- Mora, N. y Velasquez, Y. (2026). *UCampus: Plataforma de difusión universitaria* [Informe]. Universidad de Cundinamarca, Ingeniería de Software II.
- Rojas Méndez, J. C. y Rodríguez Ayala, D. A. (2024). Análisis de infoxicación en la Facultad de Ingeniería de la Universidad Distrital Francisco José de Caldas. *Revistas Udistrital*.